

Competitive Programming

Section 45: Flows and Matchings

Competitive Programming Course

2026-09-14

Table of contents

1	Flow as controlled movement	3
2	The residual graph	4
3	Dinic's algorithm	5
4	Minimum cut	6
5	Bipartite matching	7
6	Implementation and tests	8
7	Exercises	9

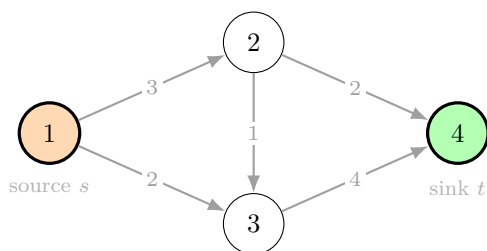
1 Flow as controlled movement

A flow network is a directed graph whose edges have capacities, plus two distinguished vertices: a source s that may create material and a sink t that may absorb it. Every other vertex is a junction that must pass on exactly what it receives.

Formally, a flow assigns a number $f(u,v)$ to each edge with

$$0 \leq f(u,v) \leq c(u,v) \quad \text{capacity}$$
$$\text{sum of flow in} = \text{sum of flow out, for } v \neq s, t \quad \text{conservation}$$

and its **value** is the net amount leaving s . Conservation means that value can equally well be measured as the amount arriving at t .



Conservation, read at vertex 2. Whatever arrives on $1 \rightarrow 2$ must leave again along $2 \rightarrow 3$ and $2 \rightarrow 4$ together. Only s may create flow, and only t may absorb it.

One network, four questions. This is the sample input of problem A, and every figure in this section uses it.

Edge labels are *capacities*. A flow puts a number $f(u,v)$ on each edge with $0 \leq f(u,v) \leq c(u,v)$, and every vertex except s and t must send out exactly what it takes in.

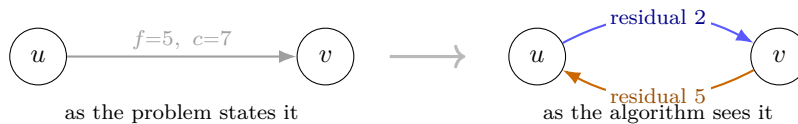
What is the largest flow $s \rightarrow t$?	problem A
Which edges form the cheapest barrier?	problem C
How large can a matching be?	problem B
Which projects should be taken?	problem D

The answer to the first is 5, and it is *not* reachable by obvious paths alone — that is what the rest of this section is about.

Four questions in this section all reduce to that one object, and all four problems use graphs of this shape.

2 The residual graph

Everything interesting about flow algorithms comes from one idea: a decision already made must remain reversible. That is what the residual graph encodes.



Forward residual $c - f = 2$: how much more can still be pushed along the edge.

Backward residual $f = 5$: how much of the flow already there may be *taken back*.

The backward edge is not a road that exists in the input. It is permission to cancel up to 5 units of an earlier decision when a better global routing turns up. Without it, an algorithm would have to get every choice right the first time; with it, no choice is ever final.

The update. Pushing x along an edge does two things at once: subtract x from the forward residual and add x to the backward one. Store the index of the reverse edge, not just the endpoint, or the second half of that update cannot be done.

Concretely, if an edge of capacity 7 currently carries 5:

forward residual	$c - f = 2$	push at most 2 more
backward residual	$f = 5$	take back at most 5

An augmenting path is any s -to- t path in the residual graph, and it may mix forward and backward edges freely. Pushing x along it subtracts x from each forward residual and adds x to each matching backward residual.

In code, each edge is stored together with the **index** of its reverse edge, so that both halves of that update are one array access apart. Storing only the reverse endpoint is not enough: parallel edges between the same pair would become indistinguishable.

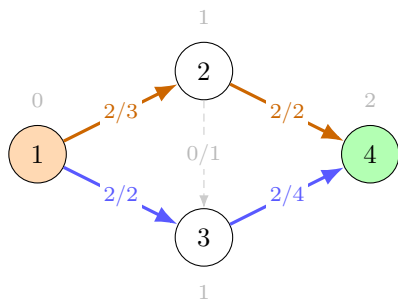
3 Dinic's algorithm

Repeatedly augmenting along any residual path terminates, but slowly. Dinic organises the search into phases:

1. BFS from s over edges with positive residual, giving $\text{level}[v]$.
2. DFS that may only move from v to to when $\text{level}[to] == \text{level}[v] + 1$.
3. Keep sending flow in this level graph until no augmenting path remains.
4. Relevel and repeat; stop when the BFS cannot reach t .

The level restriction is not a heuristic. It forbids sideways and backward moves, so the DFS cannot wander, and each phase strictly increases the length of the shortest s -to- t path. That bounds the number of phases by $O(V)$.

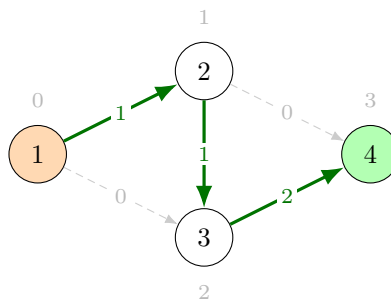
Phase 1 levels 0, 1, 1, 2



$2 \rightarrow 3$ joins two vertices of the *same* level, so it is not in the level graph at all

blocking flow = 4

Phase 2 levels 0, 1, 2, 3



releveling put 3 below 2, so $2 \rightarrow 3$ is now usable; the two saturated edges drop out

blocking flow = 1, total = 5

Phase 1 labels show *flow/capacity*; phase 2 labels show the *residual* left on each edge.

Four is not the answer. The first phase saturates both obvious routes and stops at 4, because the only edge that could carry more, $2 \rightarrow 3$, runs between two vertices of the same level. A fresh BFS then puts 3 one level below 2, the edge becomes legal, and $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ carries the fifth unit. A third BFS cannot reach 4 at all, and that is the proof that 5 is maximum.

This is the whole reason Dinic relevels instead of running one DFS: each phase strictly lengthens the shortest $s \rightarrow t$ path, so there are at most $O(V)$ phases.

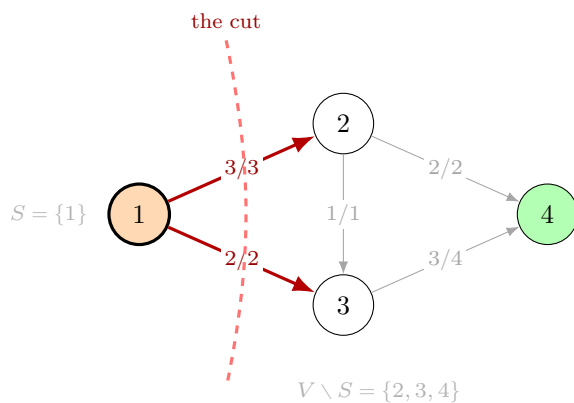
The pointer array $\text{it}[v]$, reset once per phase, remembers how many of v 's outgoing edges have already been exhausted, so no edge is examined twice within a phase.

Two habits matter more than the rest. Rebuild level every phase, and reset it at the same time; a stale pointer array silently skips usable edges.

4 Minimum cut

A cut is a split of the vertices into S containing s and $V - S$ containing t . Its capacity is the total capacity of the original edges pointing from S into $V - S$. Edges pointing back into S contribute nothing.

Any flow is bounded by any cut, since everything reaching t must cross. The max-flow min-cut theorem says the two optima coincide, and the proof is constructive: after a maximum flow, take S to be the set of vertices still reachable from s in the residual graph.



Reading the cut off the residual graph.

After the maximum flow, walk from s using only edges with residual capacity left. Here both edges out of 1 are saturated, so the walk stops immediately and $S = \{1\}$.

The cut is the set of *original* edges from S to $V \setminus S$:

- 1 $\rightarrow$ 2 of capacity 3
- 1 $\rightarrow$ 3 of capacity 2
- total **5** = the maximum flow.

Every such edge must be saturated — otherwise it would still have residual capacity and its far end would have been reached, putting it inside S . That is the whole proof.

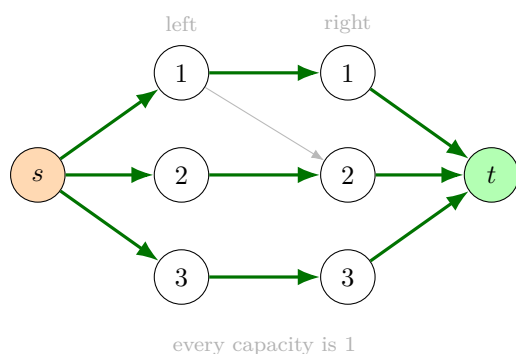
Note 3 $\rightarrow$ 4 carries 3 of 4 and is *not* saturated. It does not matter: both ends lie outside S , so it never crosses the cut. Only edges leaving S are counted, and edges re-entering S are ignored.

That set is exactly what problem C asks you to print, and it costs one extra BFS after the flow is already computed.

5 Bipartite matching

A matching is a set of edges sharing no endpoint. For a bipartite graph the maximum matching is a maximum flow on a graph built in three lines:

<code>s -> every left vertex</code>	<code>capacity 1</code>
<code>left u -> right v for each edge</code>	<code>capacity 1</code>
<code>every right vertex -> t</code>	<code>capacity 1</code>



Problem B's sample, as a flow network. Add s with an edge to every left vertex, t with an edge from every right vertex, and give *every* edge capacity 1.

One unit of flow is one matched pair. Capacity 1 out of s stops a left vertex being used twice; capacity 1 into t does the same on the right. So a maximum integral flow *is* a maximum matching, here of size **3**.

Why the residual edge matters here too. Suppose vertex 1 takes right-2 first. Then vertex 2 has nowhere to go — right-2 is its only option. The augmenting search does not give up: it walks *backwards* along the edge $1 \rightarrow \text{right-2}$, sends 1 to right-1 instead, and frees right-2 for vertex 2. That backward step is exactly the residual edge of the previous figure.

The classic augmenting-path DFS used for problem B is this same algorithm with the gadget left implicit.

Because capacities are integral, some maximum flow is integral too, so each edge carries 0 or 1 and the saturated middle edges are a matching. The capacity-1 edges at s and t are what forbid reusing a vertex.

For problem B the usual augmenting-path DFS is shorter to write than a full Dinic, and it is the same algorithm: “try to give this left vertex a partner, and if that partner is taken, ask its current owner to move”. That request to move is a backward residual edge, spelled out in plain recursion.

6 Implementation and tests

Edge storage. Keep one array of edges and, for each vertex, a list of indices into it. Adding $u \rightarrow v$ with capacity c appends two entries: the forward edge with residual c , and its reverse with residual 0. Edge i and edge $i + 1$ are then reverses of each other, which makes the update after pushing x a two-line operation.

Types. Capacities and the accumulated flow go in `long long` in C++. Total capacity out of the source can exceed 2^{31} long before n does.

Python. Write the DFS iteratively or raise the recursion limit; a long chain of unit capacities reaches recursion depth $O(V)$. Read all input at once with `sys.stdin.buffer.read().split()`.

Test	What it catches
a single edge $s \rightarrow t$	the base case, and off-by-one in vertex numbering
two disjoint paths	flow accumulating rather than overwriting
a bottleneck edge in the middle	using the wrong minimum along a path
sink unreachable from the source	an output of 0 rather than a crash
the sample of problem A	an augmenting path that needs a residual edge
parallel edges between one pair	merging edges that must stay separate
for matching, many left vertices on one right vertex	a DFS that never reassigns
for matching, $e = 0$	an empty-input path

Run the local reference suite with:

```
CP_TARGET=solution python3 sections/45_flows_and_matchings/check.py
```


7 Exercises

Use A to get the edge-and-reverse storage right, because everything else in the section is built on it. Use B to see that the augmenting-path DFS is the same algorithm with the gadget left implicit, and test it on a case where a left vertex must give up its partner. Use C to notice that the cut costs one BFS after the flow and nothing more, and that S is defined by residual capacity, not by which edges look saturated. Use D as the blind synthesis problem: the rules are implications, profits may be negative, and the whole task is deciding what a cut should mean before any code is written. Test a single edge, an unreachable sink, parallel edges, and a matching instance with no edges at all.

- A. Max Flow: Print the maximum flow from vertex 1 to vertex n .
- B. Bipartite Matching: Print the size of a maximum matching between the two sides.
- C. Min Cut Reachable Side: Print the flow value and the source side of a minimum cut.
- D. Project Selection: Choose projects under “if a then b” rules to maximize total profit.